

LINUX PROCESS CREATION, EXECUTION AND TERMINATION SYSTEM

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

CHADA HASIKA	2520030372
KEERTHANA GORANTLA	2520030168
BABBULA NIKITA	2520030172

Under the Esteemed Guidance of

Dr. K. HEMA

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Linux Process Creation, Execution and Termination System”** is a record of Bonafide work of **CHADA HASIKA - 2520030372, KEERTHANA GORANTLA - 2520030168, BABBULA NIKITA - 2520030172** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

CHADA HASIKA	2520030372
KEERTHANA GORANTLA	2520030168
BABBULA NIKITA	2520030172

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project-based report entitled **“Linux Process Creation, Execution and Termination System”** is a Bonafide work done and submitted by **CHADA HASIKA – 2520030372, KEERTHANA GORANTLA – 2520030168, BABBULA NIKITA – 2320030172** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. Anusha**, and my guide **Dr. K. Hema**, Department of Computer Science, which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the **Department for Computer Science Engineering** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

CHADA HASIKA 2520030372

KEERTHANA GORANTLA 2520030168

BABBULA NIKITA 2520030172

Abstract

The **Linux Process Creation, Execution and Termination System** is a C-based Operating Systems project developed to demonstrate the complete lifecycle of a Linux process in a simple and interactive manner. The system demonstrates how a parent process creates a child process using `fork()`, how the child can replace its program using `exec()`, and how the parent monitors and collects the child's termination status using `wait()` or `waitpid()`. It also displays the Process ID (PID) and Parent Process ID (PPID) to clearly show the relationship between processes.

The project covers important concepts of process execution, synchronization, and termination. It demonstrates both normal and abnormal termination of child processes and uses termination-status functions such as `WIFEXITED()`, `WEXITSTATUS()`, `WIFSIGNALED()`, and `WTERMSIG()` to identify how a process terminates. The system helps users understand how the parent and child processes interact during execution and how the parent can synchronize with the child and collect its status.

The system also demonstrates zombie and orphan processes, which are important aspects of Linux process management. A zombie process is demonstrated when a child terminates while the parent has not yet collected its status, while an orphan process is demonstrated when the parent terminates before the child. Overall, the project provides a practical understanding of the Linux process lifecycle, from creation and execution to synchronization and termination, helping students connect Operating Systems concepts with their implementation in a Linux environment.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	

		5.4 Module Description	
		5.5 Input and Output Design	
6	Implementation		30–45
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	Execution and Demonstration		62–67
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		79–83
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		84–86
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Process management is an important part of an Operating System. Linux uses processes to execute programs and provides system calls such as `fork()`, `exec()`, `wait()`, and `waitpid()` to manage them. However, concepts such as parent-child processes, PID, PPID, zombie processes, and orphan processes can be difficult to understand through theory alone. This project provides a practical way to observe these concepts and understand how processes are created, executed, synchronized, and terminated. The project demonstrates the complete Linux process lifecycle, from process creation using `fork()` to execution, termination, and status collection. It also demonstrates normal, abnormal, zombie, and orphan process behavior.

1.2 Problem Statement

Understanding Linux process management is an important part of Operating Systems, but concepts such as `fork()`, `exec()`, `wait()`, zombie processes, and orphan processes can be difficult to understand through theory alone. Therefore, this project aims to develop a Linux-based process lifecycle demonstration system that allows students to practically observe how processes are created, executed, synchronized, and terminated. The system also displays PID and PPID information and demonstrates normal, abnormal, zombie, and orphan process behaviour.

1.3 Scope of the Project

The scope of the Linux Process Creation, Execution and Termination System is to provide a practical demonstration of the complete lifecycle of a Linux process. The project focuses on process creation using `fork()`, program execution using `exec()`, and process synchronization using `wait()` and `waitpid()`. The system also covers PID and PPID identification, normal and abnormal process termination, zombie processes, and orphan processes. These features help users understand the relationship between parent and child processes and observe how processes behave during different stages of their lifecycle. Overall, the project is mainly intended as an educational tool for learning Linux process management and synchronization. It provides students with practical knowledge by allowing them to observe the process flow from creation to execution, termination, and status collection.

1.4 Objectives

- To create child processes using the `fork()` system call.
- To display the PID and PPID of parent and child processes.
- To demonstrate the parent-child process relationship.
- To execute a new program using `exec()`.
- To demonstrate process synchronization using `wait()` and `waitpid()`.
- To display the child's termination and exit status.
- To demonstrate normal and abnormal process termination.
- To demonstrate zombie process behavior.
- To demonstrate orphan process behavior.
- To understand the complete lifecycle of a Linux process.
- To provide students with practical knowledge of Linux process management.

1.5 Importance of the application

The Linux Process Creation, Execution and Termination System is important for understanding how processes are managed in the Linux operating system. It provides a practical way to learn how processes are created, executed, synchronized, and terminated. The application demonstrates important system calls such as `fork()`, `exec()`, `wait()`, and `waitpid()`.

The application also helps students understand the relationship between parent and child processes by displaying their PID and PPID. It demonstrates normal and abnormal termination, as well as zombie and orphan processes, which makes complex operating system concepts easier to understand.

Overall, this application provides hands-on knowledge of **Linux process management and process lifecycle**. It is useful for OS laboratory experiments, classroom demonstrations, project presentations, and viva preparation.

1.6 Target Users/Audience

The primary target users of the **Linux Process Creation, Execution and Termination System** are **students studying Operating Systems and Linux programming**. It is especially useful for undergraduate students who need practical knowledge of process management and system calls.

The application can also be used by **teachers and faculty members** to demonstrate process creation, execution, synchronization, and termination in a classroom or laboratory environment. It provides a simple and interactive way to explain concepts such as **parent and child processes, PID, PPID, zombie processes, orphan processes, and process termination**.

The system is also suitable for **beginners learning Linux system programming**, as it allows them to understand process-related concepts through practical demonstrations rather than only theoretical explanations.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Operating System Concepts Related to the Project

The Linux Process Creation, Execution and Termination System is based on several important Operating System concepts related to process management. The project provides practical demonstrations of how Linux creates, executes, synchronizes, and terminates processes.

- **Process Creation:** The `fork()` system call is used to create a new child process from the parent process.
- **Process Identification:** `getpid()` and `getppid()` are used to identify the Process ID (PID) and Parent Process ID (PPID).
- **Parent-Child Relationship:** Demonstrates how a parent process creates and manages its child process.
- **Process Execution:** The `execl()` system call replaces the current program with a new program without creating a new process.
- **Process Synchronization:** `wait()` and `waitpid()` allow the parent process to wait for the child process to complete.
- **Process Termination:** Demonstrates both normal and abnormal termination of processes.
- **Exit Status:** The parent process can obtain information about how the child process terminated.
- **Zombie Process:** Demonstrates a child process that has terminated but whose exit status has not yet been collected by its parent.
- **Orphan Process:** Demonstrates a child process whose parent has terminated before the child.
- **Process Lifecycle:** The project demonstrates the complete lifecycle of a process, from creation → execution → synchronization → termination.

2.2 Existing Methods/Algorithms

In the existing approach, Linux process management concepts are generally demonstrated individually using basic C programs and Linux system calls. These methods focus on understanding how a process is created, executed, synchronized, and terminated.

The main existing methods relevant to this project are:

1. `fork()` – Process Creation: Creates a child process from a parent process.
2. `exec()` – Program Execution: Replaces the current program with another program.
3. `wait()` / `waitpid()` – Process Synchronization: Allows the parent process to wait for the child and collect its termination status.
4. PID and PPID – Process Identification: Used to identify processes and understand the parent-child relationship.
5. Termination Status Checking: `WIFEXITED()`, `WEXITSTATUS()`, `WIFSIGNALED()`, and `WTERMSIG()` are used to determine how a child process terminated.
6. Zombie and Orphan Demonstrations: Used to understand process behavior when the parent and child processes terminate at different times.

2.3 Proposed Approach

The proposed system combines the major Linux process-management concepts into a single process lifecycle demonstration system. It creates a child process using `fork()` and displays the PID and PPID to show the parent-child relationship. The child can use `exec()` to execute another program, while the parent uses `wait()` or `waitpid()` for synchronization.

The system also demonstrates normal and abnormal process termination, along with zombie and orphan processes. This allows students to observe different process behaviors practically and understand how Linux manages processes.

The overall process flow is:

Parent → `fork()` → Child → `exec()` → Running → Termination → `wait()/waitpid()` → Status Collection.

2.4 Advantages of the Proposed Approach

- Easy to understand: Combines different process-management concepts in one system.
- Practical learning: Allows students to observe process creation, execution, synchronization, and termination.
- Clear process relationship: PID and PPID help identify parent and child processes.
- Complete lifecycle demonstration: Shows the process flow from creation to termination and status collection.
- Covers different process behaviors: Demonstrates normal, abnormal, zombie, and orphan processes.
- Educational: Helps students connect theoretical Operating Systems concepts with practical Linux implementation.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The hardware requirements for the Linux Process Creation, Execution and Termination System are:

- Processor: Intel/AMD processor or equivalent
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB of available space
- Display: Monitor or laptop display
- Keyboard: Required for entering commands and interacting with the system
- Mouse: Required for general system interaction
- Network Connection: Required for GitHub and project repository access

These requirements are sufficient for developing, compiling, and running the C-based Linux process-management programs.

3.2 Software Requirements

The following software is required to develop and run the Linux Process Creation, Execution and Termination System:

- Operating System: Ubuntu/Linux
- Programming Language: C
- Compiler: GCC (GNU Compiler Collection)
- Code Editor: Visual Studio Code / Vim
- Terminal: Linux Terminal
- Version Control: Git and GitHub
- Process Management System Calls: `fork()`, `exec()`, `wait()`, `waitpid()`
- Process Information Functions: `getpid()`, `getppid()`
- Process Status Macros: `WIFEXITED()`, `WEXITSTATUS()`, `WIFSIGNALED()`, `WTERMSIG()`
- Linux /proc Filesystem: Used for observing process-related information
- File System Calls: `open()`, `read()`, and `close()`

3.3 Operating System Environment – Ubuntu/ WSL

Ubuntu / WSL (Windows Subsystem for Linux) is used as the operating system environment for developing and executing the project. It provides a Linux-based environment where C programs can be compiled and tested using the GCC compiler and Linux terminal.

The environment supports the Linux process-management system calls required by the project, including `fork()`, `exec()`, `wait()`, and `waitpid()`. It also allows the project to demonstrate PID/PPID, normal and abnormal termination, zombie processes, and orphan processes in a practical Linux environment.

Using Ubuntu or WSL makes it possible to observe how Linux creates, executes, synchronizes, and terminates processes, helping connect the theoretical concepts of operating systems with their actual implementation.

3.4 Development Tools and Compilers

The project uses the following development tools and compiler for implementation and testing:

- GCC (GNU Compiler Collection): Used to compile the C source code into executable programs.
- Visual Studio Code (VS Code): Used as the primary code editor for writing and managing C programs.
- Vim: Can also be used as a terminal-based editor for editing source files in Linux.
- Linux Terminal: Used to compile, execute, and test the process-management programs.
- Git: Used for version control and managing project changes.
- GitHub: Used to store and manage the project source code repository.
- C Programming Language: Used to implement the process creation, execution, synchronization, and termination modules.

These tools provide the required environment for developing, compiling, executing, and testing the Linux Process Creation, Execution and Termination System.

CHAPTER – 4 TECHNOLOGY AND TOOLS USED

4.1 C/C++ Programming

C/C++ programming is used to develop the project and implement the Linux process-management concepts. The programs demonstrate important operations such as process creation, execution, synchronization, and termination.

For this project, C programming is mainly used with Linux system calls such as `fork()`, `exec()`, `wait()`, and `waitpid()`. C is suitable because it provides direct access to Linux system-level functions and makes it easier to demonstrate the complete process lifecycle.

The programs are compiled using the GCC compiler and executed in the Ubuntu/WSL environment.

4.2 GCC Compiler

GCC (GNU Compiler Collection) is used to compile the C programs developed for the project. It converts the C source code into executable files that can be run in the Ubuntu/WSL environment.

GCC supports the Linux system calls and functions used in the project, such as `fork()`, `exec()`, `wait()`, and `waitpid()`. It is used to compile, execute, and test the different process-management modules of the Linux Process Creation, Execution and Termination System.

4.3 Linux System Calls

Linux system calls are used in the project to interact with the operating system and manage processes. They provide the basic operations required to demonstrate the complete process lifecycle.

The main system calls and functions used in the project are:

- `fork()` – Creates a new child process.
- `exec()` – Replaces the child process program with another program.
- `wait()` – Allows the parent process to wait for a child to terminate.
- `waitpid()` – Allows the parent to wait for a specific child process.
- `getpid()` – Returns the process ID (PID) of the current process.
- `getppid()` – Returns the process ID of the parent process.

4.4 Linux Commands and Utilities

Linux commands and utilities are used to compile, execute, monitor, and test the process-management programs in the project. They provide a convenient way to interact with the Ubuntu/WSL environment and observe process-related information.

Some useful commands and utilities include:

- `gcc` – Compiles the C programs.
- `./program` – Executes the compiled program.
- `ps` – Displays information about running processes.
- `top` – Monitors active processes and system resources.
- `kill` – Sends signals to a process.
- `echo` – Displays text or values in the terminal.
- `man` – Provides documentation for Linux commands and system calls.

4.5 Other Tools / Libraries Used

The project uses the following supporting tools and libraries:

- `Git` and `GitHub`: Used for version control and maintaining the project source code.
- `VS Code` / `Vim`: Used for writing and editing C source files.
- `Linux Terminal`: Used to compile, execute, and test the programs.
- `Standard C Library`: Provides basic functions required for C programming.
- `POSIX/Linux APIs`: Provide access to process-management operations and system-level functionality.
- `/proc Filesystem`: Can be used to observe process-related information in Linux.
- `GCC`: Used to compile the C source code into executable programs.

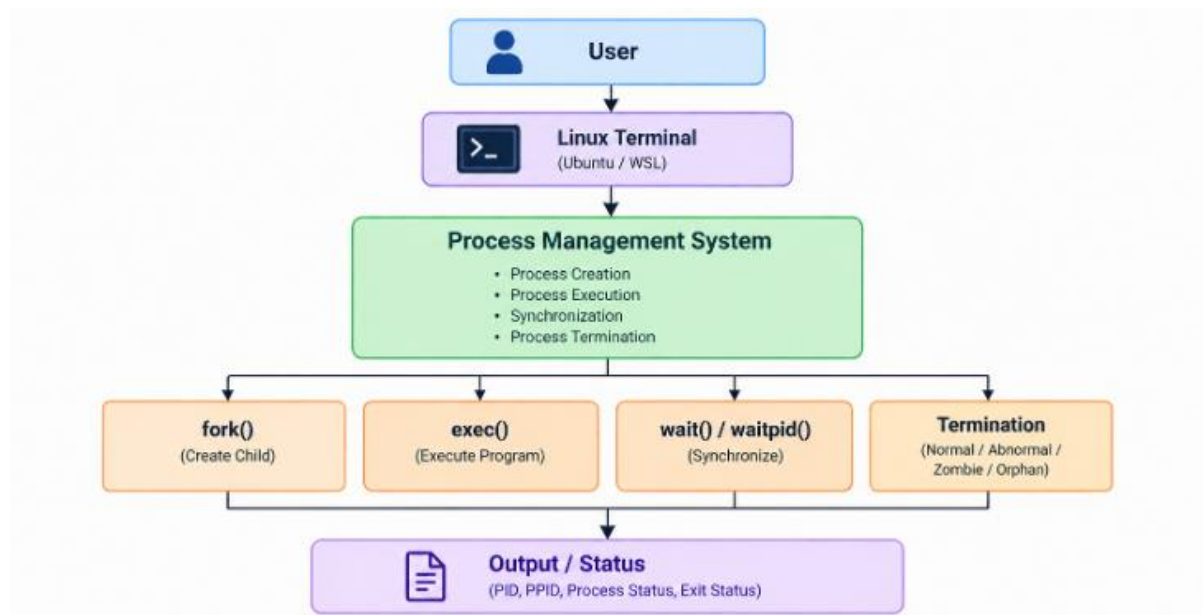
CHAPTER -5 SYSTEM ARCHITECTURE AND

5.1 Overall System Architecture

The Linux Process Creation, Execution and Termination System follows a simple process-management architecture. The user interacts with the program through the Linux terminal, and the main program acts as the parent process.

The parent process uses `fork()` to create a child process. The child can use `exec()` to execute another program, while the parent uses `wait()` or `waitpid()` to synchronize with the child and collect its termination status.

The system also contains modules for normal termination, abnormal termination, zombie processes, and orphan processes. The PID and PPID are displayed to show the relationship between parent and child processes.



5.2 System Workflow

- The system workflow represents the complete lifecycle of a Linux process:
- The user starts the program through the Linux terminal.
- The parent process is created and its PID is displayed.
- The parent calls `fork()` to create a child process.
- The child process displays its PID and PPID.
- The child uses `exec()` to execute another program when required.

- The parent uses wait() or waitpid() to synchronize with the child.
- The child terminates either normally or abnormally.
- The parent checks and collects the termination status.
- The system can demonstrate zombie and orphan processes.
- The final process status is displayed to the user.

Start



Parent Process



fork()



Child Created



Display PID / PPID



exec()



Program Execution



Process Termination



wait() / waitpid()

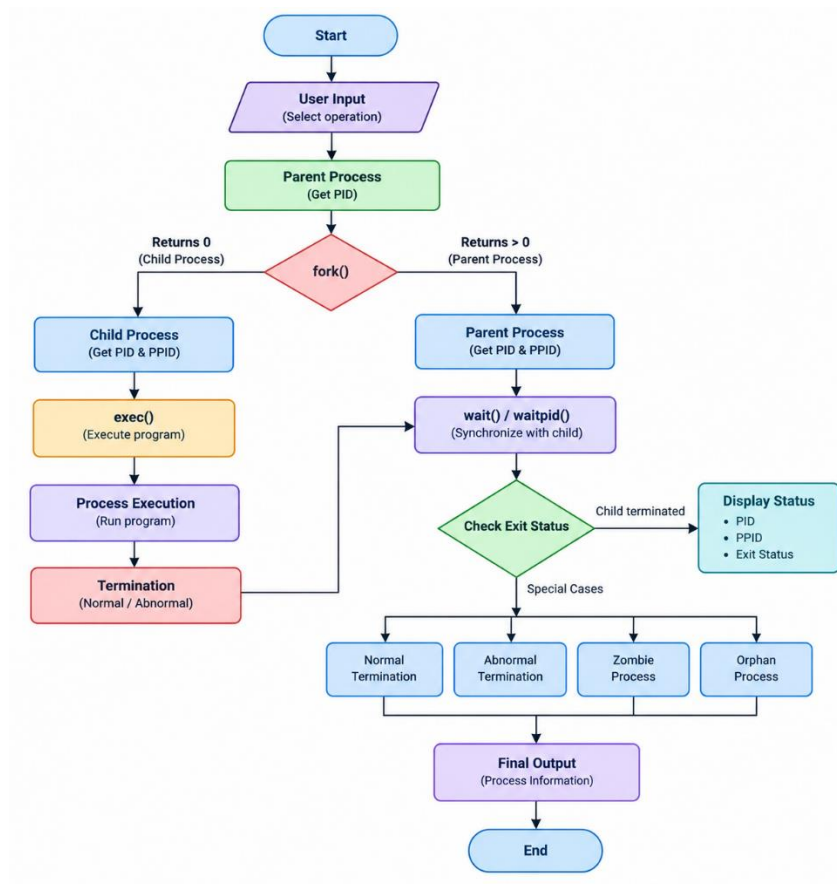


Status Collection



End

5.3 Flowchart / Data Flow



5.4 Module Description

Module	Description
Process Creation Module	Uses fork() to create a child process.
PID/PPID Module	Displays the process ID and parent process ID using getpid() and getppid().
ProcessExecution Module	Uses exec() to replace the child process with another program.
Synchronization Module	Uses wait() and waitpid() to synchronize the parent and child.
NormalTermination Module	Demonstrates successful process termination and collects the exit status.
AbnormalTermination Module	Demonstrates termination caused by a signal and checks the termination status.

Module	Description
Zombie Process Module	Demonstrates a terminated child whose status has not yet been collected by the parent.
Orphan Process Module	Demonstrates a child continuing after its parent process terminates.
Lifecycle Display Module	Presents the complete process lifecycle from creation to termination.

5.5 Input and Output Design

Input Design

The system accepts input from the user through the Linux terminal. The user can select or execute different process-management demonstrations such as:

- Process creation using `fork()`
- Program execution using `exec()`
- Normal termination
- Abnormal termination
- Zombie process
- Orphan process

Output Design

The output is displayed in the Linux terminal. It provides process-related information such as:

- Parent PID
- Child PID
- Parent PPID
- Process execution status
- Normal termination status
- Abnormal termination/signal information
- Zombie/orphan process behavior

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

Module	Implementation
Process Creation	Uses fork() to create a child process.
PID/PPID Display	Uses getpid() to obtain process identification information.
Program Execution	Uses exec() to replace the child process program.
Synchronization	Uses wait() and waitpid() to synchronize the parent and child.
Normal Termination	Checks successful child termination and exit status.
Abnormal Termination	Demonstrates signal-based child termination.
Zombie Process	Child terminates while the parent does not immediately call wait().
Orphan Process	Parent terminates before the child, allowing the child to continue.
Lifecycle Display	Shows the complete process sequence from creation to status collection.

6.2 Algorithm / Pseudocode

START

1. Start the parent process.
2. Display the parent PID.
3. Call fork().
4. Check the result of fork().
5. If fork() returns 0:
 - a. Child process is created.
 - b. Display child PID and PPID.
 - c. Execute another program using exec().
 - d. Perform the required operation.
 - e. Terminate the child.
6. If fork() returns a positive value:
 - a. Parent process continues.
 - b. Wait for the child using wait() or waitpid().

- c. Collect the child's termination status.
 - d. Check whether termination was normal or abnormal.
7. Demonstrate zombie/orphan behavior when selected.
8. Display the process information.
- END

6.3 Core C / C++ Implementation

The project is primarily a C-based implementation. C is used to interact with Linux process-management system calls and functions.

- `fork()`
- `exec()`
- `wait()`
- `waitpid()`
- `getpid()`
- `getppid()`

Termination status is handled using macros such as:

- `WIFEXITED()`
- `WEXITSTATUS()`
- `WIFSIGNALED()`
- `WTERMSIG()`

6.4 Linux System Calls / APIs Used

System Call / API	Purpose
<code>fork()</code>	Creates a child process.
<code>exec()</code>	Replaces the current program with another program.
<code>wait()</code>	Waits for a child process to terminate.
<code>waitpid()</code>	Waits for a specific child process.
<code>getpid()</code>	Obtains the process ID.

System Call / API	Purpose
getppid()	Obtains the parent process ID.
WIFEXITED()	Checks whether a child terminated normally.
WEXITSTATUS()	Obtains the child's exit status.
WIFSIGNALED()	Checks whether a child terminated because of a signal.
WTERMSIG()	Identifies the signal that terminated the child.
/proc	Provides process and system information.
open(), read(), close()	Used to read information from Linux system interfaces.

6.5 Process / Thread Implementation

Process Implementation

The main focus of the project is Linux process implementation rather than multithreading.

The parent process creates a child using `fork()`. The child process can execute another program using `exec()`, while the parent uses `wait()` or `waitpid()` to synchronize with the child. PID and PPID information is displayed to demonstrate the parent-child relationship.

The project also demonstrates:

- Parent process
- Child process
- Normal termination
- Abnormal termination
- Zombie process
- Orphan process

6.6 IPC / Synchronization Implementation

Synchronization

Synchronization is implemented using:

- `wait()`
- `waitpid()`

The parent process uses these functions to wait for the child process and collect its termination status. This ensures that the parent can monitor the completion of the child process.

6.7 Optional User Interface Implementation

The project provides a simple interactive environment for observing process creation, execution, synchronization, and termination.

The current interface can be represented as a Linux terminal-based interface, where the user runs the C program and observes:

- Process creation
- PID and PPID
- Program execution
- Process termination
- Exit status
- Zombie process behavior
- Orphan process behavior

CHAPTER -7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

The project is compiled in the Ubuntu/WSL environment using the GCC compiler. The C source file is compiled into an executable file using:

```
gcc program.c -o program
```

7.2 Execution Commands

After compilation, the program can be executed from the Linux terminal using:

```
./program
```

For example:

```
$ gcc process.c -o process
```

```
$ ./process
```

The program then displays the process-management options and allows the user to execute the required process demonstration.

7.3 Sample Inputs and Outputs

```
===== Linux Process Management =====
```

1. Process Creation
2. Process Execution
3. Normal Termination
4. Abnormal Termination
5. Zombie Process
6. Orphan Process
7. Exit

Enter your choice: 1

Parent Process PID : 2450

Child Process Created

Child PID : 2451

Parent PID : 2450

7.4 Demonstration of OS Concepts

OS Concept	Demonstration
Process Creation	fork() creates a child process.
Parent-Child Relationship	PID and PPID are displayed.
Program Execution	exec() replaces the child program.
Process Synchronization	wait() and waitpid() synchronize parent and child.
Normal Termination	WIFEXITED() and WEXITSTATUS() identify normal termination and exit status.
Abnormal Termination	WIFSIGNALED() and WTERMSIG() identify signal-based termination.
Zombie Process	Child terminates while the parent has not immediately collected its status.
Orphan Process	Parent terminates before the child, allowing the child to continue.
Process Lifecycle	Parent → fork() → Child → exec() → Running → Termination → wait()/waitpid() → Status Collection.

CHAPTER - 8 CONCLUSION

8.1 Summary of the project

The Linux Process Creation, Execution and Termination System is a C-based Operating Systems project developed to provide a practical understanding of the Linux process lifecycle. The project demonstrates how a parent process creates a child process using `fork()`, how the child can execute another program using `exec()`, and how the parent synchronizes with the child using `wait()` and `waitpid()`.

The system also displays the Process ID (PID) and Parent Process ID (PPID) to clearly demonstrate the relationship between parent and child processes. Different process termination conditions are demonstrated, including normal termination and abnormal termination.

Another important part of the project is the demonstration of zombie and orphan processes. These examples help users understand what happens when a child process terminates before its parent collects its status and when a parent terminates before its child. Overall, the project provides an interactive and practical way to understand process creation, execution, synchronization, and termination in Linux.

8.2 Objectives Achieved

The project successfully achieved the major objectives defined during its development:

1. Process Creation: Child processes were created using the `fork()` system call.
2. PID and PPID Identification: PID and PPID information was displayed for understanding process relationships.
3. Parent-Child Relationship: The relationship between parent and child processes was demonstrated.
4. Program Execution: The `exec()` mechanism was used to demonstrate execution of another program.
5. Process Synchronization: `wait()` and `waitpid()` were used for parent-child synchronization.
6. Termination Status: The child's termination and exit status were collected and displayed.

7. Abnormal Termination: Signal-based abnormal termination was demonstrated.
8. Zombie Process: The behavior of a child remaining after termination before status collection was demonstrated.
9. Orphan Process: The behavior of a child continuing after its parent terminates was demonstrated.
10. Complete Lifecycle: The complete sequence from process creation to termination and status collection was demonstrated.
11. Practical Learning: The project provided practical knowledge of Linux process management.

8.3 Results Achieved

The implementation provides a practical demonstration of the major stages of a Linux process lifecycle. The system successfully demonstrates process creation, program execution, synchronization, termination, and status collection.

The use of PID and PPID makes the parent-child relationship easier to observe. The project also demonstrates how `wait()` and `waitpid()` allow the parent process to monitor the child and obtain its termination information.

The project further demonstrates different process states and termination conditions, including normal termination, abnormal termination, zombie processes, and orphan processes. These demonstrations help bridge the gap between theoretical Operating Systems concepts and their practical implementation in Linux.

The expected outcome described in the project document is therefore achieved: students can observe process creation, execution, synchronization, termination, PID/PPID information, and different process behaviors in a Linux environment.

8.4 Skills and learning outcomes

The project helped the team develop practical skills in Operating Systems, Linux programming, and system-level C programming.

Technical Skills

- Understanding Linux process management
- Using `fork()` for process creation
- Using `exec()` for program execution
- Using `wait()` and `waitpid()` for synchronization
- Working with PID and PPID
- Understanding process termination status
- Understanding normal and abnormal termination
- Demonstrating zombie and orphan processes
- Compiling programs using GCC
- Working with the Ubuntu/Linux terminal
- Understanding Linux `/proc` and system information interfaces

Practical Learning Outcomes

The project helped the team understand how theoretical concepts such as process creation, parent-child relationships, synchronization, and termination work in an actual Linux environment. It also improved skills in testing, debugging, system-level programming, and project integration.

CHAPTER – 9 REFERENCES

9.1 Books

The following books can be used as supporting references for the Operating Systems concepts discussed in the project:

1. Silberschatz, A., Galvin, P. B., and Gagne, G., *Operating System Concepts*, Wiley.
2. Tanenbaum, A. S. and Bos, H., *Modern Operating Systems*, Pearson.
3. Stevens, W. R. and Rago, S. A., *Advanced Programming in the UNIX Environment*, Addison-Wesley.

These references provide background information about process management, process creation, synchronization, system calls, and Unix/Linux operating systems.

9.2 Linux / GNU Documentation

Linux and GNU documentation provides technical information required to understand and implement the system calls and development tools used in the project.

1. Linux Manual Pages (man pages) – Used for understanding Linux commands, system calls, and process-management functions.
2. GNU Compiler Collection (GCC) Documentation – Used as a reference for compiling C programs.
3. GNU C Library Documentation – Provides information about C library functions and Linux system interfaces.
4. Linux /proc Documentation – Provides information about accessing process and system information.

The project document specifically identifies Linux/Ubuntu, GCC, C/C++, Git/GitHub, and VS Code/Vim as the development tools and platforms.

9.3 Online Resources

The following online resources can be included in the final report:

1. Linux Man Pages – Reference for fork(), exec(), wait(), waitpid(), process IDs, and process termination.
2. GNU GCC Documentation – Reference for compiling and building C programs.
3. Ubuntu Documentation – Reference for the Ubuntu/Linux development environment.
4. GitHub – Used for source-code repository management.
5. Linux /proc Documentation – Reference for process and system information.

Project Repository

The submitted project document provides the following GitHub repository for the project:

Linux Process Creation, Execution and Termination System – OS
<https://github.com/chadahasika27-sketch/Linux-Process-Creation-Execution-and-Termination-System-OS>

This repository contains the project source code and supports version control and collaborative development.